

**Departamento de Engenharia Informática**

## **Jogo ISOLA em Prolog com Algoritmo Alfa-Beta**

51620 : Radesh Govind (A51620@alunos.isel.pt)

51619 : Martim Monteiro (A51619@alunos.isel.pt)

Relatório para a Unidade Curricular de Inteligência Artificial  
da Licenciatura em Engenharia Informática e de Computadores

Professor : Professor Nuno Leite



# Resumo

---

Este trabalho documenta a implementação do jogo de estratégia *ISOLA* em SWI-Prolog, no âmbito da unidade curricular de Inteligência Artificial. O jogo foi representado através de listas de listas em Prolog, com predicados para geração de movimentos, aplicação de jogadas e detecção de fim de jogo. A inteligência artificial do jogador computadorizado baseia-se no algoritmo *alfa-beta*, uma implementação eficiente do princípio *minimax*, complementada com ordenação heurística de movimentos. A função de avaliação combina mobilidade e centralidade. Foram realizados testes unitários (33 testes, todos bem sucedidos) e benchmarks experimentais em tabuleiros de dimensões e profundidades variadas. Os resultados confirmam que a profundidade 4 é praticável em tabuleiros 3×3, mas exige tempo elevado em tabuleiros 6×6 (cerca de 2 min 47 s no primeiro movimento), justificando a profundidade 2 como compromisso para tabuleiros maiores.



# Abstract

---

This report documents the implementation of the ISOLA strategy board game in SWI-Prolog, developed as part of the Artificial Intelligence course. The game state is represented as a list of lists in Prolog, with predicates for move generation, move application, and end-game detection. The computer player's artificial intelligence is based on the alpha-beta algorithm, an efficient implementation of the minimax principle, enhanced with heuristic move ordering. The evaluation function combines mobility and board centrality. Unit testing comprised 33 tests, all passing. Experimental benchmarks across varying board sizes and search depths show that depth 4 is practical on  $3 \times 3$  boards (136 ms), but becomes prohibitive on  $6 \times 6$  boards (approximately 167 s for the first move), making depth 2 the recommended compromise for larger boards.



# Índice

---

|                                                                    |             |
|--------------------------------------------------------------------|-------------|
| <b>Lista de Figuras</b>                                            | <b>xi</b>   |
| <b>Lista de Tabelas</b>                                            | <b>xiii</b> |
| <b>Lista de Listagens</b>                                          | <b>xv</b>   |
| <b>1 Introdução</b>                                                | <b>1</b>    |
| 1.1 O Jogo ISOLA . . . . .                                         | 1           |
| 1.1.1 Regras do Jogo . . . . .                                     | 1           |
| 1.1.2 Exemplo de Jogo . . . . .                                    | 1           |
| 1.2 Objetivos de Aprendizagem . . . . .                            | 2           |
| 1.3 Introdução ao Prolog . . . . .                                 | 2           |
| 1.3.1 Analogia: O Detetive e as Pistas . . . . .                   | 2           |
| 1.3.2 Termos e Unificação . . . . .                                | 3           |
| 1.3.3 Backtracking . . . . .                                       | 3           |
| 1.3.4 Listas em Prolog . . . . .                                   | 4           |
| 1.4 Estrutura do Relatório . . . . .                               | 4           |
| <b>2 Implementação</b>                                             | <b>5</b>    |
| 2.1 Representação do Tabuleiro ( <code>board.pl</code> ) . . . . . | 5           |
| 2.1.1 Estrutura de Dados . . . . .                                 | 5           |
| 2.1.2 Predicados Principais . . . . .                              | 6           |
| 2.2 Regras do Jogo ( <code>rules.pl</code> ) . . . . .             | 6           |
| 2.2.1 Validação de Movimentos . . . . .                            | 6           |
| 2.2.2 Aplicação de Movimentos e Remoção . . . . .                  | 7           |
| 2.2.3 Condição de Fim de Jogo . . . . .                            | 7           |
| 2.3 Inteligência Artificial ( <code>ai.pl</code> ) . . . . .       | 8           |
| 2.3.1 O Princípio Minimax . . . . .                                | 8           |

|          |                                                                    |           |
|----------|--------------------------------------------------------------------|-----------|
| 2.3.1.1  | Conceito e Analogia . . . . .                                      | 8         |
| 2.3.1.2  | Funcionamento . . . . .                                            | 8         |
| 2.3.1.3  | Valores Terminais . . . . .                                        | 9         |
| 2.3.2    | Algoritmo Alfa-Beta . . . . .                                      | 9         |
| 2.3.2.1  | Conceito e Analogia . . . . .                                      | 9         |
| 2.3.2.2  | Os Parâmetros $\alpha$ e $\beta$ . . . . .                         | 9         |
| 2.3.2.3  | Implementação . . . . .                                            | 10        |
| 2.3.2.4  | Ganho de Desempenho . . . . .                                      | 11        |
| 2.3.3    | Ordenação de Movimentos . . . . .                                  | 11        |
| 2.3.4    | Função de Avaliação . . . . .                                      | 11        |
| 2.4      | Interface de Jogo (game.pl) . . . . .                              | 12        |
| 2.4.1    | Fluxo do Programa . . . . .                                        | 12        |
| 2.4.2    | Turno Humano vs. Turno IA . . . . .                                | 13        |
| 2.5      | Utilização de Inteligência Artificial no Desenvolvimento . . . . . | 13        |
| <b>3</b> | <b>Avaliação Experimental</b>                                      | <b>15</b> |
| 3.1      | Testes Unitários . . . . .                                         | 15        |
| 3.1.1    | Execução e Resultados . . . . .                                    | 15        |
| 3.1.2    | Cobertura dos Testes . . . . .                                     | 16        |
| 3.1.3    | Exemplo de Teste Unitário . . . . .                                | 16        |
| 3.2      | Benchmarks de Desempenho . . . . .                                 | 17        |
| 3.2.1    | Metodologia . . . . .                                              | 17        |
| 3.2.2    | Resultados . . . . .                                               | 18        |
| 3.2.3    | Análise dos Resultados . . . . .                                   | 18        |
| 3.2.3.1  | Fator de Ramificação Real . . . . .                                | 18        |
| 3.2.3.2  | Impacto da Profundidade . . . . .                                  | 18        |
| 3.2.3.3  | Consistência dos Movimentos Escolhidos . . . . .                   | 18        |
| 3.2.3.4  | Avaliação da Função Heurística . . . . .                           | 19        |
| 3.3      | Limitações e Trabalho Futuro . . . . .                             | 19        |
| 3.3.1    | Limitações Identificadas . . . . .                                 | 19        |
| 3.3.2    | Recomendações de Configuração . . . . .                            | 19        |
| 3.4      | Conclusões . . . . .                                               | 20        |
|          | <b>Referências</b>                                                 | <b>21</b> |



|                                           |          |
|-------------------------------------------|----------|
| <b>A Código Fonte</b>                     | <b>i</b> |
| A.1 board.pl — Tabuleiro . . . . .        | i        |
| A.2 rules.pl — Regras do Jogo . . . . .   | i        |
| A.3 ai.pl — Algoritmo Alfa-Beta . . . . . | ii       |



# Lista de Figuras

---

|     |                                                                                                                                                                                     |    |
|-----|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----|
| 1.1 | Evolução de um jogo ISOLA 6×6. O símbolo # representa uma casa permanentemente removida. . . . .                                                                                    | 2  |
| 2.1 | Movimentos válidos a partir de posições diferentes (X = destino válido, P = peão). . . . .                                                                                          | 7  |
| 2.2 | Exemplo de árvore minimax com 2 níveis. O Jogador MAX escolhe o ramo com valor máximo; o Jogador MIN escolhe o ramo com valor mínimo. O valor na raiz é 3. . . . .                  | 8  |
| 2.3 | Corte alfa: o Jogador MAX já tem garantido o valor 3 (ramo esquerdo). No ramo direito, o MIN encontrou 2, que é pior para MAX. Os restantes filhos do ramo direito são descartados. | 10 |



# Lista de Tabelas

---

|     |                                                               |    |
|-----|---------------------------------------------------------------|----|
| 2.1 | Estrutura de módulos do projeto . . . . .                     | 5  |
| 2.2 | Complexidade: minimax vs. alfa-beta . . . . .                 | 11 |
| 3.1 | Distribuição e cobertura dos testes unitários . . . . .       | 17 |
| 3.2 | Tempo de cálculo do primeiro movimento da IA . . . . .        | 18 |
| 3.3 | Valores da função de avaliação em posições de teste . . . . . | 19 |



# Lista de Listagens

---

|      |                                                            |    |
|------|------------------------------------------------------------|----|
| 1.1  | Factos e regras básicas em Prolog . . . . .                | 2  |
| 1.2  | Exemplos de unificação . . . . .                           | 3  |
| 1.3  | Demonstração de backtracking . . . . .                     | 3  |
| 1.4  | findall recolhe todas as soluções . . . . .                | 4  |
| 1.5  | Tabuleiro 3×3 como lista de listas . . . . .               | 4  |
| 2.1  | Criação do tabuleiro inicial . . . . .                     | 6  |
| 2.2  | Atualização não destrutiva de uma célula . . . . .         | 6  |
| 2.3  | Validação de um movimento . . . . .                        | 6  |
| 2.4  | Aplica um movimento e remove uma casa . . . . .            | 7  |
| 2.5  | Detecção de fim de jogo . . . . .                          | 7  |
| 2.6  | Implementação do minimax puro . . . . .                    | 8  |
| 2.7  | Valores terminais . . . . .                                | 9  |
| 2.8  | Pesquisa alfa-beta — caso recursivo . . . . .              | 10 |
| 2.9  | Função de pontuação para ordenação de movimentos . . . . . | 11 |
| 2.10 | Função de avaliação heurística . . . . .                   | 12 |
| 2.11 | Ciclo principal do jogo . . . . .                          | 13 |
| 3.1  | Teste: movimentos do centro de um tabuleiro 3×3 . . . . .  | 16 |
| A.1  | board.pl — criação e manipulação do tabuleiro . . . . .    | i  |
| A.2  | rules.pl — validação e aplicação de movimentos . . . . .   | i  |
| A.3  | ai.pl — ponto de entrada e pesquisa alfa-beta . . . . .    | ii |













# Capítulo 1

## Introdução

---

### 1.1 O Jogo ISOLA

*ISOLA* (também conhecido como *Isolation* ou *Solitude*) é um jogo de estratégia para dois jogadores criado por Bernd Kienitz em 1972 [2]. O objetivo é simples: isolar o peão do adversário de modo a que ele não consiga realizar nenhum movimento válido no seu turno.

#### 1.1.1 Regras do Jogo

O tabuleiro é uma grelha quadrada (normalmente  $6 \times 6$ ). Cada jogador possui um peão que começa em cantos opostos do tabuleiro:

- **Jogador 1** (x) começa na posição (linha 0, coluna 0) — canto superior esquerdo.
- **Jogador 2** (o) começa na posição (linha 5, coluna 5) — canto inferior direito.

Em cada turno, um jogador executa obrigatoriamente **duas ações**:

1. **Mover** o seu peão para uma casa adjacente livre (horizontal, vertical ou diagonalmente — tal como um rei no xadrez, mas apenas um passo de cada vez).
2. **Remover** qualquer casa vazia do tabuleiro (exceto as casas ocupadas pelos peões), marcando-a como permanentemente inacessível (#).

**Não é permitido** saltar por cima do adversário nem entrar em casas já removidas. Vence o jogador que fizer com que o adversário não tenha qualquer movimento válido no início do seu turno.

#### 1.1.2 Exemplo de Jogo

O exemplo abaixo mostra a evolução de um jogo numa grelha  $6 \times 6$ . No estado inicial, o Jogador 1 move-se de (0,0) para (1,1) e remove a casa (2,2).

A estratégia consiste em manter o próprio peão com muitas opções de movimento (**mobilidade**) enquanto se reduz progressivamente a mobilidade do adversário, acabando por o isolar completamente.

| Estado inicial |        |   |   |   |   |   | Após P1 mover (0,0)→(1,1), remover (2,2) |        |   |   |   |   |   |
|----------------|--------|---|---|---|---|---|------------------------------------------|--------|---|---|---|---|---|
|                | 0      | 1 | 2 | 3 | 4 | 5 |                                          | 0      | 1 | 2 | 3 | 4 | 5 |
|                | +----- |   |   |   |   |   |                                          | +----- |   |   |   |   |   |
| 0              |        | x | . | . | . | . | 0                                        |        | . | . | . | . | . |
| 1              |        | . | . | . | . | . | 1                                        |        | . | x | . | . | . |
| 2              |        | . | . | . | . | . | 2                                        |        | . | . | # | . | . |
| 3              |        | . | . | . | . | . | 3                                        |        | . | . | . | . | . |
| 4              |        | . | . | . | . | . | 4                                        |        | . | . | . | . | . |
| 5              |        | . | . | . | . | o | 5                                        |        | . | . | . | . | o |

Figura 1.1: Evolução de um jogo ISOLA 6×6. O símbolo # representa uma casa permanentemente removida.

## 1.2 Objetivos de Aprendizagem

A realização deste trabalho visa consolidar os seguintes objetivos:

1. Compreender e dominar a sintaxe de programas Prolog.
2. Utilizar construções básicas: átomos, termos compostos, listas, operadores e aritmética.
3. Compreender e controlar o *backtracking*.
4. Saber utilizar predicados embutidos típicos (`findall/3`, `nth0/3`, `maplist/3`, etc.).
5. Implementar um jogo de duas pessoas com informação perfeita.
6. Compreender o princípio *minimax*.
7. Compreender o algoritmo *alfa-beta*: uma implementação eficiente do *minimax*.

## 1.3 Introdução ao Prolog

Para compreender a implementação, é essencial entender os fundamentos do Prolog [1, 5]. Ao contrário de linguagens como Java ou Python, o Prolog não descreve como calcular um resultado, mas sim o que é verdade. O computador trata de descobrir como chegar lá.

### 1.3.1 Analogia: O Detetive e as Pistas

Imagine que o Prolog é um detetive. Você fornece-lhe um conjunto de **factos** (pistas conhecidas) e **regras** (deduções lógicas), e depois faz-lhe **perguntas** (consultas). O detetive procura sistematicamente todas as combinações possíveis até encontrar uma resposta.

```

1 % Factos: quem gosta de que
2 gosta(maria, gatos).
3 gosta(joao, caes).
4 gosta(maria, caes).
5

```

```

6 % Regra: duas pessoas tem algo em comum
7 em_comum(X, Y, Coisa) :-
8     gosta(X, Coisa),
9     gosta(Y, Coisa),
10    X \= Y.
11
12 % Consulta: ?- em_comum(maria, joao, Coisa).
13 % Resposta: Coisa = caes.

```

Listagem 1.1: Factos e regras básicas em Prolog

A linha que começa com `:-` lê-se como “se”: `em_comum(X, Y, Coisa)` é verdade **se** `gosta(X, Coisa)` é verdade **e** `gosta(Y, Coisa)` é verdade **e** `X` não é igual a `Y`.

### 1.3.2 Termos e Unificação

O elemento central do Prolog é o **termo**. Existem quatro tipos:

- **Átomo** — constante simbólica: `gato`, `x`, `'.'`, `true`.
- **Número** — inteiro ou vírgula flutuante: `3`, `4.5`.
- **Variável** — começa com maiúscula ou sublinhado: `X`, `Jogador`, `_`.
- **Termo composto** — funtor com argumentos: `move(1/2, 3/4)`, `[a,b,c]`.

**Unificação** é o mecanismo pelo qual o Prolog associa valores a variáveis. Quando se escreve `X = 3`, o Prolog unifica `X` com `3`. A unificação só tem sucesso se os dois termos tiverem a mesma estrutura.

```

1 ?- X = 5.           % X = 5   (sucesso)
2 ?- 2/3 = R/C.       % R = 2, C = 3 (decomposicao de termos)
3 ?- [H|T] = [1,2,3]. % H = 1, T = [2,3] (cabeca e cauda)
4 ?- 2 = 3.           % false (nao unificam)

```

Listagem 1.2: Exemplos de unificação

### 1.3.3 Backtracking

O *backtracking* é o mecanismo de busca do Prolog. Quando uma tentativa de satisfazer um objetivo falha, o Prolog **recua** automaticamente e tenta a próxima alternativa disponível.

**Analogia:** imagine um labirinto. O Prolog tenta um caminho; se chegar a um beco sem saída, volta atrás (*backtrack*) e tenta o próximo corredor, até explorar todos os caminhos.

```

1 cor(vermelho).
2 cor(verde).
3 cor(azul).
4
5 % Consulta: ?- cor(X).

```

```

6 % X = vermelho      (primeira solucao)
7 % X = verde         (apos pedir mais: ";")
8 % X = azul          (terceira solucao)
9 % false             (sem mais solucoes)

```

Listagem 1.3: Demonstração de backtracking

O predicado `findall/3` aproveita o *backtracking* para recolher todas as soluções numa lista, o que é amplamente usado neste projeto:

```

1 ?- findall(X, cor(X), Lista).
2 % Lista = [vermelho, verde, azul]

```

Listagem 1.4: `findall` recolhe todas as soluções

### 1.3.4 Listas em Prolog

Em Prolog, uma lista é representada como `[Elem1, Elem2, ...]`. A notação `[H|T]` separa a cabeça (H) da cauda (T).

No jogo ISOLA, o tabuleiro é uma **lista de listas**: cada linha do tabuleiro é uma lista de átomos.

```

1 Tabuleiro = [
2   [x, '.', '.'], % linha 0
3   ['.', '.', '.'], % linha 1
4   ['.', '.', o]   % linha 2
5 ]
6 % Simbolos: '.' = livre, x = piao P1,
7 %           o = piao P2, '#' = removida

```

Listagem 1.5: Tabuleiro 3×3 como lista de listas

## 1.4 Estrutura do Relatório

O presente relatório está organizado da seguinte forma:

- **Capítulo 2 — Implementação:** Descreve em detalhe todos os módulos do projeto: representação do tabuleiro, regras do jogo, algoritmos de inteligência artificial e interface de jogo.
- **Capítulo 3 — Avaliação Experimental:** Apresenta os resultados dos testes unitários e os benchmarks de desempenho obtidos em diferentes cenários.
- **Apêndice A — Código Fonte:** Listagens selecionadas dos ficheiros Prolog desenvolvidos.



## Capítulo 2

# Implementação

---

Este capítulo descreve a implementação do jogo ISOLA em SWI-Prolog. O projeto está organizado em cinco módulos:

Tabela 2.1: Estrutura de módulos do projeto

| Ficheiro | Responsabilidade                     | Predicados principais             |
|----------|--------------------------------------|-----------------------------------|
| board.pl | Representação e display do tabuleiro | initial_board, get_cell, set_cell |
| rules.pl | Regras e validação de movimentos     | valid_move, apply_move, game_over |
| ai.pl    | Inteligência artificial (alfa-beta)  | best_move, alphabeta, evaluate    |
| game.pl  | Interface e ciclo de jogo            | play, game_loop, human_turn       |
| main.pl  | Ponto de entrada                     | initialization                    |

## 2.1 Representação do Tabuleiro (board.pl)

### 2.1.1 Estrutura de Dados

O tabuleiro é representado como uma **lista de listas** em Prolog. Cada elemento da lista exterior é uma linha do tabuleiro; cada elemento de uma linha é um átomo que indica o conteúdo da casa:

- ' .' — casa livre (vazia)
- x — peão do Jogador 1
- o — peão do Jogador 2
- ' #' — casa permanentemente removida

As posições são representadas como o termo composto `Linha/Coluna` (indexação a partir de zero). Por exemplo, `2/3` significa linha 2, coluna 3.

**Vantagem desta representação:** as listas são a estrutura nativa do Prolog, e predicados como `nth0/3` (acesso por índice) e `maplist/2` (aplicação funcional) estão disponíveis na biblioteca padrão, o que simplifica enormemente o código.

### 2.1.2 Predicados Principais

O predicado `initial_board/2` cria um tabuleiro de tamanho  $N \times N$  totalmente preenchido com casas livres:

```
1 initial_board(Size, Board) :-
2     length(Board, Size),           % Board tem Size linhas
3     maplist(make_row(Size), Board). % cada linha tem Size colunas
4
5 make_row(Size, Row) :-
6     length(Row, Size),
7     maplist(=('.'), Row).          % cada célula é '.'
```

Listagem 2.1: Criação do tabuleiro inicial

**Como funciona:** `length(Board, Size)` cria uma lista de `Size` variáveis não instanciadas. O `maplist` aplica `make_row` a cada elemento, e dentro de cada linha, `maplist(=('.'), Row)` unifica cada célula com o átomo `'.'`.

A atualização de uma célula é **não destrutiva**: o Prolog não modifica estruturas existentes, mas constrói uma nova versão do tabuleiro. Esta é uma característica fundamental da programação lógica.

```
1 set_cell(Board, Row, Col, Value, NewBoard) :-
2     nth0(Row, Board, OldRow, RestRows), % separa a linha Row
3     nth0(Col, OldRow, _, RestCols),      % separa o elemento Col
4     nth0(Col, NewRow, Value, RestCols),  % reconstrói linha com novo valor
5     nth0(Row, NewBoard, NewRow, RestRows). % reconstrói tabuleiro
```

Listagem 2.2: Atualização não destrutiva de uma célula

**Analogia:** pense num fotograma de cinema. Quando um personagem se move, não se apaga o fotograma anterior — cria-se um novo fotograma com o personagem na nova posição. O Prolog funciona da mesma forma.

## 2.2 Regras do Jogo (rules.pl)

### 2.2.1 Validação de Movimentos

Um movimento é válido se a casa de destino:

1. estiver a exatamente um passo (em qualquer das 8 direções);
2. estiver dentro dos limites do tabuleiro;
3. estiver livre `'.'`.

```
1 valid_move(Board, FromR, FromC, ToR, ToC) :-
2     board_size(Board, Size),
3     Max is Size - 1,
```

```

4      member(DR, [-1, 0, 1]),      % delta de linha: -1, 0 ou +1
5      member(DC, [-1, 0, 1]),      % delta de coluna: -1, 0 ou +1
6      \+ (DR == 0, DC == 0),      % nao pode ficar no lugar
7      ToR is FromR + DR,
8      ToC is FromC + DC,
9      ToR >= 0, ToR <= Max,        % dentro dos limites
10     ToC >= 0, ToC <= Max,
11     get_cell(Board, ToR, ToC, '.'). % casa livre

```

Listagem 2.3: Validação de um movimento

Este predicado funciona tanto como **verificador** (quando ToR/ToC já estão instanciados) como **gerador** (quando estão livres, o Prolog gera por *backtracking* todas as combinações de DR/DC). Esta dualidade é uma das características mais elegantes do Prolog.

| Peão no centro: 8 movimentos | Peão no canto: 3 movimentos |
|------------------------------|-----------------------------|
| X X X                        | P X .                       |
| X P X                        | X X .                       |
| X X X                        | . . .                       |

Figura 2.1: Movimentos válidos a partir de posições diferentes (X = destino válido, P = peão).

## 2.2.2 Aplicação de Movimentos e Remoção

```

1 % Move o peao de From para To
2 apply_move(Board, FromR, FromC, ToR, ToC, NewBoard) :-
3     get_cell(Board, FromR, FromC, Piece), % le o simbolo
4     set_cell(Board, FromR, FromC, '.', B1), % limpa origem
5     set_cell(B1, ToR, ToC, Piece, NewBoard). % coloca no destino
6
7 % Marca uma casa como permanentemente removida
8 apply_remove(Board, Row, Col, NewBoard) :-
9     set_cell(Board, Row, Col, '#', NewBoard).

```

Listagem 2.4: Aplica um movimento e remove uma casa

## 2.2.3 Condição de Fim de Jogo

O jogo termina quando o jogador cujo turno é jogar não tem nenhum movimento válido:

```

1 game_over(Board, Player, P1Pos, P2Pos) :-
2     player_pos(Player, P1Pos, P2Pos, CR/CC),
3     \+ valid_move(Board, CR, CC, _, _).
4     % \+ significa negacao: true se valid_move FALHA

```

Listagem 2.5: Detecção de fim de jogo

O operador `\+` é a **negação por falha** do Prolog: “`\+ Goal`” é verdade se e só se `Goal` falha. Aqui, `game_over` é verdade se não existe nenhum movimento válido.

## 2.3 Inteligência Artificial (ai.pl)

### 2.3.1 O Princípio Minimax

#### 2.3.1.1 Conceito e Analogia

O *minimax* é o algoritmo clássico para jogos de dois jogadores com informação perfeita [4].

**Analogia:** imagine que você está a jogar xadrez e quer decidir o melhor movimento. Pensa: “Se eu jogar aqui, o meu adversário vai responder com o seu melhor movimento, e depois eu vou responder com o meu melhor, e assim por diante.” O *minimax* formaliza exatamente este raciocínio.

#### 2.3.1.2 Funcionamento

O algoritmo constrói mentalmente uma **árvore de jogo**:

- **Nós MAX** (Jogador 1, símbolo x): escolhe o movimento que **maximiza** a pontuação.
- **Nós MIN** (Jogador 2, símbolo o): escolhe o movimento que **minimiza** a pontuação (prejudica o Jogador 1).
- **Nós folha** (profundidade 0 ou jogo terminado): a posição é avaliada por uma função heurística.

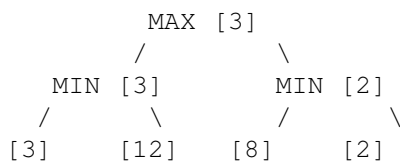


Figura 2.2: Exemplo de árvore minimax com 2 níveis. O Jogador MAX escolhe o ramo com valor máximo; o Jogador MIN escolhe o ramo com valor mínimo. O valor na raiz é 3.

```

1 % Caso base 1: profundidade esgotada -> avaliacao estatica
2 minimax(Board, _Player, P1Pos, P2Pos, 0, Value) :-
3     !, evaluate(Board, P1Pos, P2Pos, Value).
4
5 % Caso base 2: jogador sem movimentos -> derrota terminal
6 minimax(Board, Player, P1Pos, P2Pos, _D, Value) :-
7     player_pos(Player, P1Pos, P2Pos, CR/CC),
8     \+ valid_move(Board, CR, CC, _, _), !,
9     terminal_value(Player, Value).
10
11 % Caso recursivo: gera filhos e aplica max/min
12 minimax(Board, Player, P1Pos, P2Pos, Depth, Value) :-
13     all_move_pairs(Board, Player, P1Pos, P2Pos, Pairs),
14     NextD is Depth - 1,
15     Other is 3 - Player,      % 1->2, 2->1
  
```

```

16  maplist(minimax_child(Board, Player, Other,
17              P1Pos, P2Pos, NextD),
18              Pairs, ChildValues),
19  ( Player == 1
20  -> max_list(ChildValues, Value) % MAX escolhe o maior
21  ; min_list(ChildValues, Value) % MIN escolhe o menor
22  ) .

```

Listagem 2.6: Implementação do minimax puro

### 2.3.1.3 Valores Terminais

Quando um jogador fica sem movimentos, o jogo termina:

```

1 terminal_value(1, -1000). % Jogador 1 preso -> derrota (negativo)
2 terminal_value(2, 1000). % Jogador 2 preso -> vitoria de P1 (positivo)

```

Listagem 2.7: Valores terminais

Os valores  $\pm 1000$  são intencionalmente muito maiores do que qualquer valor de avaliação heurística, garantindo que uma vitória ou derrota real é sempre preferida a um estado com boa heurística.

## 2.3.2 Algoritmo Alfa-Beta

### 2.3.2.1 Conceito e Analogia

O *minimax* é correto mas lento: explora todos os nós da árvore. O *alfa-beta* é uma otimização que descarta ramos que **nunca podem influenciar o resultado**, sem alterar o valor final calculado.

**Analogia:** imagine que está a seleccionar o melhor restaurante numa lista. Já visitou um restaurante com classificação 8/10. Ao visitar o segundo restaurante, descobre que tem um prato horrível de 3/10. Sem sequer provar o resto da ementa, sabe que este restaurante nunca será a sua primeira escolha. **Pode ignorar o resto da visita** e passar ao próximo. O alfa-beta faz exatamente o mesmo com ramos da árvore de jogo.

### 2.3.2.2 Os Parâmetros $\alpha$ e $\beta$

- $\alpha$  (*alpha*) — o melhor valor que o Jogador MAX já garantiu. Começa em  $-\infty$ .
- $\beta$  (*beta*) — o melhor valor que o Jogador MIN já garantiu. Começa em  $+\infty$ .
- **Corte beta** (em nó MAX): se o valor atual  $\geq \beta$ , o Jogador MIN nunca escolheria este ramo. Pode parar.
- **Corte alfa** (em nó MIN): se o valor atual  $\leq \alpha$ , o Jogador MAX nunca escolheria este ramo. Pode parar.

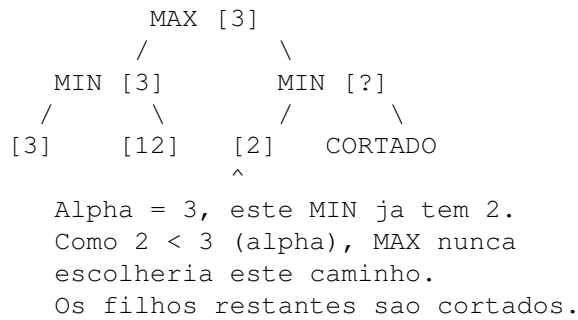


Figura 2.3: Corte alfa: o Jogador MAX já tem garantido o valor 3 (ramo esquerdo). No ramo direito, o MIN encontrou 2, que é pior para MAX. Os restantes filhos do ramo direito são descartados.

### 2.3.2.3 Implementação

```

1 alphabeta(Board, Player, P1Pos, P2Pos, Depth, Alpha, Beta,
2     Value, BM, BR) :-
3     all_move_pairs(Board, Player, P1Pos, P2Pos, Pairs),
4     order_pairs(Pairs, Board, Player, P1Pos, P2Pos, Ordered),
5     NextDepth is Depth - 1,
6     Other is 3 - Player,
7     ( Player == 1
8     -> ab_max(Ordered, Board, Other, P1Pos, P2Pos,
9         Player, NextDepth, Alpha, Beta,
10        -10000, none, none, Value, BM, BR)
11    ;   ab_min(Ordered, Board, Other, P1Pos, P2Pos,
12        Player, NextDepth, Alpha, Beta,
13        10000, none, none, Value, BM, BR)
14    ).
15
16 % No MAX: actualiza alpha e corta se alpha >= beta
17 ab_max([move(MPos,RPos)|Rest], Board, Other, P1Pos, P2Pos,
18     CurPlayer, Depth, Alpha, Beta, BestVal, BestM, BestR,
19     Val, BM, BR) :-
20     apply_pair(Board, CurPlayer, P1Pos, P2Pos, MPos, RPos,
21         NewBoard, NewP1, NewP2),
22     alphabeta(NewBoard, Other, NewP1, NewP2, Depth,
23         Alpha, Beta, ChildVal, _, _),
24     ( (BestM = none ; ChildVal > BestVal)
25     -> NewBest = ChildVal, NewBM = MPos, NewBR = RPos
26     ;   NewBest = BestVal, NewBM = BestM, NewBR = BestR
27     ),
28     NewAlpha is max(Alpha, NewBest),
29     ( NewAlpha >= Beta
30     -> Val = NewBest, BM = NewBM, BR = NewBR % corte beta
31     ;   ab_max(Rest, Board, Other, P1Pos, P2Pos,
32         CurPlayer, Depth, NewAlpha, Beta,

```

```

33         NewBest, NewBM, NewBR, Val, BM, BR)
34     ) .

```

Listagem 2.8: Pesquisa alfa-beta — caso recursivo

### 2.3.2.4 Ganho de Desempenho

O corte alfa-beta reduz drasticamente o número de nós avaliados [3]:

Tabela 2.2: Complexidade: minimax vs. alfa-beta

| Algoritmo       | Complexidade | Nós (b=8, d=4) |
|-----------------|--------------|----------------|
| Minimax puro    | $O(b^d)$     | $8^4 = 4.096$  |
| Alfa-beta ideal | $O(b^{d/2})$ | $8^2 = 64$     |

Na prática, considerando que cada movimento tem em média 8 destinos possíveis e 34 casas removíveis num tabuleiro 6×6 inicial, o fator de ramificação real é  $\approx 102$  (ver Capítulo 3). O alfa-beta com boa ordenação reduz este fator para  $\approx \sqrt{102} \approx 10$  por nível.

### 2.3.3 Ordenação de Movimentos

A eficácia do corte alfa-beta depende fortemente da ordem em que os movimentos são avaliados. Avaliar primeiro os movimentos mais prometedores maximiza os cortes.

**Heurística implementada:**

- **Movimentos centrais primeiro:** casas perto do centro do tabuleiro oferecem mais opções futuras (maior mobilidade).
- **Remoções próximas do adversário primeiro:** remover casas adjacentes ao adversário reduz mais rapidamente a sua mobilidade.

```

1 score_pair(Center, OppR, OppC,
2             move(MR/MC, RR/RC),
3             Score-move(MR/MC, RR/RC)) :-
4     MDR is abs(MR - Center), MDC is abs(MC - Center),
5     MoveScore is MDR + MDC,      % menor = mais central
6     RDR is abs(RR - OppR), RDC is abs(RC - OppC),
7     RemScore is RDR + RDC,      % menor = mais perto do adversario
8     Score is MoveScore * 3 - RemScore. % peso: movimento > remocao

```

Listagem 2.9: Função de pontuação para ordenação de movimentos

### 2.3.4 Função de Avaliação

Quando a pesquisa atinge a profundidade máxima (sem o jogo ter terminado), é necessário avaliar heurísticamente a posição atual.

**Dois critérios foram escolhidos:**

1. **Mobilidade** — o número de movimentos legais disponíveis. Um jogador com mais movimentos tem mais liberdade e está mais longe de ser isolado. Este critério tem peso 10.
2. **Centralidade** — quão próximo está o peão do centro do tabuleiro. O centro oferece mais direções de fuga. Este critério serve para desempatar.

```

1 evaluate(Board, P1Pos, P2Pos, Value) :-
2     P1Pos = P1R/P1C, P2Pos = P2R/P2C,
3     valid_moves(Board, P1R, P1C, M1), length(M1, S1), % mobilidade P1
4     valid_moves(Board, P2R, P2C, M2), length(M2, S2), % mobilidade P2
5     board_size(Board, Size),
6     Center is (Size - 1) / 2.0,
7     centrality(P1R, P1C, Center, Cent1),
8     centrality(P2R, P2C, Center, Cent2),
9     Value is (S1 - S2) * 10 + (Cent1 - Cent2).
10    % positivo = bom para P1; negativo = bom para P2

```

Listagem 2.10: Função de avaliação heurística

**Exemplo prático:** num tabuleiro 3×3 com P1 no canto (0,0) e P2 no centro (1,1), o cálculo é:

- Mobilidade P1 = 2 movimentos (confinado ao canto)
- Mobilidade P2 = 7 movimentos (centro, máxima liberdade)
- Diferença de mobilidade:  $(2 - 7) \times 10 = -50$
- Centralidade:  $(0 + 0) - (1 + 1) = -2$  (P2 está mais ao centro)
- Pontuação total:  $-50 + (-2) = -52$

O valor negativo indica que P2 está em clara vantagem — o que é correto: P2 no centro tem muito mais liberdade.

**Justificação da escolha:** a mobilidade é o indicador mais direto de quanto um jogador está próximo de ser isolado. A centralidade captura a tendência estratégica de manter o peão no centro do tabuleiro.

## 2.4 Interface de Jogo (game.pl)

### 2.4.1 Fluxo do Programa

O jogo segue o seguinte fluxo:

1. play/0 — exibe o menu principal.
2. main\_menu/0 — permite escolher o modo de jogo (humano-humano, humano-IA, IA-IA).
3. setup\_game/2 — configura o tamanho do tabuleiro e a profundidade de pesquisa da IA.
4. start\_game/4 — cria o tabuleiro inicial com os peões nos cantos opostos.



5. `game_loop/6` — ciclo principal: exibe o tabuleiro, verifica fim de jogo, executa o turno do jogador corrente.

```

1 game_loop(Board, Turn, P1Pos, P2Pos, Mode, Depth) :-
2     display_game(Board, Turn),
3     ( game_over(Board, Turn, P1Pos, P2Pos)
4     -> Winner is 3 - Turn, % o adversario venceu
5         announce_winner(Winner, Board),
6         play_again
7     ; execute_turn(Board, Turn, P1Pos, P2Pos,
8                     Mode, Depth,
9                     NewBoard, NewP1, NewP2),
10        NextTurn is 3 - Turn, % alterna: 1->2, 2->1
11        game_loop(NewBoard, NextTurn, NewP1, NewP2,
12                  Mode, Depth)
13    ).

```

Listagem 2.11: Ciclo principal do jogo

### 2.4.2 Turno Humano vs. Turno IA

Quando o turno pertence a um jogador humano, o programa pede a entrada no formato `Linha/Coluna`. (com ponto final, pois o SWI-Prolog interpreta a entrada como um termo Prolog completo):

```

Move (linha/coluna, ex: 1/2): 1/1.
Remove uma casa (linha/coluna): 2/2.

```

Quando o turno pertence à IA, o predicado `best_move/7` é invocado automaticamente, que executa o alfa-beta e devolve o par (movimento, remoção) ótimo.

## 2.5 Utilização de Inteligência Artificial no Desenvolvimento

Ao longo do desenvolvimento deste projeto, a IA generativa (nomeadamente o modelo Claude da Anthropic) foi utilizada de forma extensa e integrada em todas as fases do trabalho. A sua utilização não se limitou a gerar código, mas abrangeu todo o processo de aprendizagem e desenvolvimento:

1. **Aprendizagem de conceitos:** a IA foi consultada para compreender os fundamentos do Prolog (unificação, *backtracking*, listas, predicados embutidos) através de explicações, exemplos e analogias.
2. **Compreensão dos algoritmos:** o princípio *minimax* e o algoritmo *alfa-beta* foram explorados com a assistência da IA, que forneceu explicações passo a passo, diagramas de árvores e análise de complexidade.
3. **Exploração de alternativas:** foram discutidas diferentes abordagens para a representação do tabuleiro (lista de listas vs. tabela de hash), para a função de avaliação (mobilidade, centralidade, distância entre peões) e para a ordenação de movimentos.

4. **Escrita e depuração de código:** a IA auxiliou na escrita de predicados, na correção de erros lógicos (incluindo um *bug* na acumulação de valores no alfa-beta), e na formulação dos 33 testes unitários.
5. **Elaboração do relatório:** a IA foi utilizada para auxiliar na redação, estruturação e revisão deste documento, com o objetivo de tornar as explicações claras e acessíveis.

Em todos os casos, o uso da IA foi orientado pelos objetivos didáticos da unidade curricular: a IA serviu como ferramenta de aprendizagem e produtividade, e não como substituto da compreensão dos conceitos.

## Capítulo 3

# Avaliação Experimental

---

Este capítulo apresenta os resultados dos testes realizados ao sistema: testes unitários de correcção funcional e benchmarks de desempenho em diferentes configurações de tabuleiro e profundidade de pesquisa.

### 3.1 Testes Unitários

O projeto inclui uma suite de 33 testes unitários (ficheiro `tests/tests.pl`), organizados em quatro grupos.

#### 3.1.1 Execução e Resultados

Os testes foram executados com o comando:

```
swipl -g "run_tests, halt" tests/tests.pl
```

O resultado obtido foi:

```
% Start unit: board
% [1/33] board:initial_board_6x6_size ..... passed (0.015 sec)
% [2/33] board:initial_board_6x6_row_length ..... passed (0.001 sec)
% [3/33] board:initial_board_all_free ..... passed (0.000 sec)
% [4/33] board:all_cells_3x3_count ..... passed (0.000 sec)
% [5/33] board:get_cell_after_set ..... passed (0.000 sec)
% [6/33] board:set_cell_nondestructive ..... passed (0.000 sec)
% [7/33] board:get_cell_oob_row ..... passed (0.000 sec)
% [8/33] board:get_cell_oob_col ..... passed (0.000 sec)
% [9/33] board:empty_cell_free ..... passed (0.000 sec)
% [10/33] board:empty_cell_pawn ..... passed (0.000 sec)
% [11/33] board:empty_cell_removed ..... passed (0.000 sec)
% End unit board: passed (0.007 sec CPU)
```

```

% Start unit: rules
% [12/33] rules:valid_moves_centre_3x3_count ..... passed (0.000 sec)
% [13/33] rules:valid_moves_corner_3x3_count ..... passed (0.000 sec)
% [14/33] rules:valid_moves_blocked_by_remove ..... passed (0.000 sec)
% [15/33] rules:valid_moves_no_overlap_opponent ..... passed (0.000 sec)
% [16/33] rules:valid_removes_excludes_pawns ..... passed (0.000 sec)
% [17/33] rules:valid_removes_excl_removed_cell ..... passed (0.000 sec)
% [18/33] rules:apply_move_source_cleared ..... passed (0.000 sec)
% [19/33] rules:apply_remove_marks_cell ..... passed (0.000 sec)
% [20/33] rules:game_over_no_moves ..... passed (0.000 sec)
% [21/33] rules:game_over_has_moves ..... passed (0.000 sec)
% [22/33] rules:player_pos_p1 ..... passed (0.000 sec)
% [23/33] rules:player_pos_p2 ..... passed (0.000 sec)
% End unit rules: passed (0.001 sec CPU)

% Start unit: ai
% [24/33] ai:evaluate_mobility_advantage ..... passed (0.000 sec)
% [25/33] ai:evaluate_opponent_advantage ..... passed (0.000 sec)
% [26/33] ai:evaluate_symmetric ..... passed (0.000 sec)
% [27/33] ai:terminal_value_p1_loses ..... passed (0.000 sec)
% [28/33] ai:terminal_value_p2_wins ..... passed (0.000 sec)
% [29/33] ai:best_move_trivial_win ..... passed (0.001 sec)
% [30/33] ai:alphabeta_terminal ..... passed (0.000 sec)
% [31/33] ai:minimax_depth_zero ..... passed (0.000 sec)
% End unit ai: passed (0.002 sec CPU)

% Start unit: integration
% [32/33] integration:scripted_turn_1 ..... passed (0.000 sec)
% [33/33] integration:scripted_game_over_detection .. passed (0.000 sec)
% End unit integration: passed (0.000 sec CPU)

% All 33 tests passed in 0.029 seconds (0.014 cpu)

```

Todos os 33 testes passaram em 0.029 segundos. A execução rápida deve-se ao facto de a maioria dos testes operar sobre tabuleiros 3×3 ou posições artificiais construídas especificamente para o teste.

### 3.1.2 Cobertura dos Testes

### 3.1.3 Exemplo de Teste Unitário

Para ilustrar a estrutura dos testes, apresenta-se o teste que verifica o número de movimentos válidos a partir do centro de um tabuleiro 3×3:

```

1 :- use_module(library(plunit)).
2 :- begin_tests(rules).
3

```

Tabela 3.1: Distribuição e cobertura dos testes unitários

| Módulo      | Núm. testes | Aspetos verificados                                                                                                                                                                                                               |
|-------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| board       | 11          | Criação do tabuleiro; acesso por coordenadas; atualização não destrutiva; acesso fora dos limites (deve falhar sem erro); <code>empty_cell</code> para células livres, ocupadas e removidas.                                      |
| rules       | 12          | Movimentos válidos do centro e do canto; bloqueio por casa removida; ausência de sobreposição com o adversário; lista de remoções (exclui peões e casas já removidas); aplicação de movimento e remoção; detecção de fim de jogo. |
| ai          | 8           | Função de avaliação (vantagem P1, vantagem P2, posição simétrica); valores terminais; melhor movimento em situação de vitória trivial; alfa-beta em posição terminal; minimax a profundidade zero.                                |
| integration | 2           | Turno completo (movimento + remoção num jogo real); detecção de fim de jogo no contexto de um jogo jogado.                                                                                                                        |

```

4 test(valid_moves_centre_3x3_count) :-
5     initial_board(3, B0),
6     set_cell(B0, 0, 0, x, B1),
7     set_cell(B1, 2, 2, o, Board),
8     valid_moves(Board, 1, 1, Moves), % piao no centro (1,1)
9     length(Moves, N),
10    assertion(N == 7). % 8 direcoes - 1 (canto ocupado)
11
12 :- end_tests(rules).

```

Listagem 3.1: Teste: movimentos do centro de um tabuleiro 3×3

## 3.2 Benchmarks de Desempenho

Para avaliar o desempenho do algoritmo alfa-beta, foram medidos os tempos de execução do primeiro movimento da IA em diferentes combinações de tamanho de tabuleiro e profundidade de pesquisa. Todos os testes partiram do estado inicial do jogo (peões nos cantos opostos, tabuleiro completamente livre).

### 3.2.1 Metodologia

Os tempos foram medidos com o predicado `get_time/1` do SWI-Prolog (tempo de relógio de parede), no seguinte ambiente:

- Sistema operativo: macOS Darwin 25.3.0 (Apple Silicon)
- SWI-Prolog (versão padrão, sem otimizações adicionais)
- Medida: tempo para calcular o primeiro movimento da IA a partir do estado inicial

### 3.2.2 Resultados

Tabela 3.2: Tempo de cálculo do primeiro movimento da IA

| Tabuleiro | Profundidade | Tempo                           | Movimento   | Remoção |
|-----------|--------------|---------------------------------|-------------|---------|
| 3×3       | 2            | 4 ms                            | (0,0)→(1,1) | (1,2)   |
| 3×3       | 4            | 136 ms                          | (0,0)→(1,1) | (1,2)   |
| 6×6       | 2            | 87 ms                           | (0,0)→(1,1) | (4,4)   |
| 6×6       | 4            | 167.619 ms $\approx$ 2 min 47 s | (0,0)→(1,1) | (4,4)   |

### 3.2.3 Análise dos Resultados

#### 3.2.3.1 Fator de Ramificação Real

No estado inicial de um tabuleiro 6×6, o Jogador 1 pode:

- **Mover** para 3 casas (canto com 3 vizinhos livres).
- **Remover** qualquer uma de 34 casas livres (36 casas – 2 peões).

Total de pares (movimento, remoção):  $3 \times 34 = 102$ .

Este é o fator de ramificação no nível 1. Nas jogadas seguintes, quando os peões se movem para o centro, cada jogador pode ter até 8 destinos possíveis, o que eleva o fator de ramificação.

#### 3.2.3.2 Impacto da Profundidade

O salto de profundidade 2 para profundidade 4 no tabuleiro 6×6 representa um aumento de 87 ms para 167.619 ms — um fator de aproximadamente  $1927\times$ . Este crescimento super-exponencial confirma o carácter combinatório do problema.

Embora o alfa-beta reduza significativamente o número de nós explorados em relação ao minimax puro, o fator de ramificação elevado ( $\approx 102$ ) no estado inicial de um tabuleiro 6×6 torna a profundidade 4 impraticável para uso interativo.

#### 3.2.3.3 Consistência dos Movimentos Escolhidos

Nota-se que em todas as configurações, a IA escolheu o mesmo movimento: mover o peão do canto (0,0) para o centro (1,1). Isto é consistente com a função de avaliação que favorece a centralidade — o peão no centro (1,1) de um tabuleiro 3×3 tem acesso a até 7 casas, enquanto no canto (0,0) tem apenas 3.

A diferença na escolha da remoção entre profundidade 2 ((1,2) em 3×3) e profundidade 4 ((4,4) em 6×6) reflete a visão de futuro mais profunda da pesquisa a maior profundidade.

### 3.2.3.4 Avaliação da Função Heurística

Os testes confirmaram os valores esperados da função de avaliação:

Tabela 3.3: Valores da função de avaliação em posições de teste

| Posição                        | Configuração         | Mob. P1 | Mob. P2 | Valor |
|--------------------------------|----------------------|---------|---------|-------|
| Simétrica (cantos opostos)     | 3×3, (0,0) vs. (2,2) | 3       | 3       | 0     |
| Assimétrica (canto vs. centro) | 3×3, (0,0) vs. (1,1) | 2       | 7       | −52   |

O valor 0 na posição simétrica confirma que a função não é tendenciosa para nenhum jogador. O valor −52 na posição assimétrica reflete corretamente a vantagem significativa de P2 (maior mobilidade e melhor centralidade).

## 3.3 Limitações e Trabalho Futuro

### 3.3.1 Limitações Identificadas

1. **Desempenho em tabuleiros grandes a profundidade alta:** A combinação 6×6 + profundidade 4 é proibitiva ( $\approx 167$  s por movimento). O fator de ramificação elevado, resultante da grande quantidade de casas removíveis, é a causa principal.
2. **Formato de entrada:** os jogadores humanos têm de introduzir posições no formato `Linha/Coluna`. (com ponto final), pois o predicado `read/1` do SWI-Prolog espera um termo Prolog completo terminado por ponto. Este formato pode causar confusão.
3. **Ausência de tabela de transposição:** posições iguais atingidas por caminhos diferentes são reavaliadas. Uma tabela de transposição reduziria o trabalho redundante.
4. **Sem aprofundamento iterativo:** a profundidade é fixa. O aprofundamento iterativo (aumentar gradualmente a profundidade até ao limite de tempo) melhoraria a qualidade do jogo em tempo fixo.

### 3.3.2 Recomendações de Configuração

Com base nos resultados experimentais:

- Para tabuleiros 3×3: profundidade 4 é adequada (136 ms por movimento).
- Para tabuleiros 6×6: profundidade 2 é o compromisso recomendado (87 ms por movimento), oferecendo uma resposta quase instantânea.
- Profundidade 6 não é recomendada para tabuleiros maiores que 3×3 sem otimizações adicionais.

## 3.4 Conclusões

Este trabalho demonstrou que é possível implementar um jogo de estratégia completo com IA em Prolog de forma elegante e eficaz.

### Objetivos cumpridos:

- Implementação completa do jogo ISOLA com todos os modos (humano-humano, humano-IA, IA-IA) e tamanhos de tabuleiro configuráveis.
- Algoritmo alfa-beta funcional com ordenação de movimentos e função de avaliação heurística.
- Suite de 33 testes unitários, todos bem sucedidos.
- Compreensão prática do *backtracking*, da programação lógica e dos algoritmos de pesquisa em jogos.

### Principais aprendizagens:

- O Prolog permite exprimir regras de jogo de forma quase direta a partir da descrição em linguagem natural, aproveitando o *backtracking* para gerar automaticamente todos os movimentos válidos.
- O alfa-beta é essencial para tornar o *minimax* prático: sem poda, profundidade 4 num tabuleiro 6×6 seria completamente inviável.
- A ordenação de movimentos é tão importante quanto o próprio alfa-beta: maus movimentos avaliados primeiro reduzem drasticamente a eficácia dos cortes.



# Referências

---

- [1] Ivan Bratko, *Prolog Programming for Artificial Intelligence*, Third. Addison-Wesley, 2001.
- [2] Bernd Kienitz, *ISOLA — Isolation Strategy Board Game*, Abstract strategy game for two players, 1972.
- [3] Donald E. Knuth & Ronald W. Moore, “An analysis of alpha-beta pruning”, *Artificial Intelligence*, vol. 6, n.º 4, páginas 293–326, 1975.
- [4] Stuart Russell & Peter Norvig, *Artificial Intelligence: A Modern Approach*, Fourth. Pearson, 2020.
- [5] Leon Sterling & Ehud Shapiro, *The Art of Prolog: Advanced Programming Techniques*, Second. MIT Press, 1994.



# Apêndice A

## Código Fonte

---

Este apêndice apresenta os predicados centrais de cada módulo do projeto. O código completo encontra-se no repositório do projeto.

### A.1 board.pl — Tabuleiro

```
1 initial_board(Size, Board) :-
2     length(Board, Size),
3     maplist(make_row(Size), Board).
4
5 make_row(Size, Row) :-
6     length(Row, Size),
7     maplist(=('.'), Row).
8
9 get_cell(Board, Row, Col, Cell) :-
10    board_size(Board, Size),
11    Row >= 0, Row < Size, Col >= 0, Col < Size,
12    nth0(Row, Board, RowList),
13    nth0(Col, RowList, Cell).
14
15 set_cell(Board, Row, Col, Value, NewBoard) :-
16    nth0(Row, Board, OldRow, RestRows),
17    nth0(Col, OldRow, _, RestCols),
18    nth0(Col, NewRow, Value, RestCols),
19    nth0(Row, NewBoard, NewRow, RestRows).
20
21 empty_cell('.').
```

Listagem A.1: board.pl — criação e manipulação do tabuleiro

### A.2 rules.pl — Regras do Jogo

```
1 valid_move(Board, FromR, FromC, ToR, ToC) :-
2     board_size(Board, Size), Max is Size - 1,
3     member(DR, [-1,0,1]), member(DC, [-1,0,1]),
4     \+ (DR == 0, DC == 0),
```

```

5   ToR is FromR + DR, ToC is FromC + DC,
6   ToR >= 0, ToR =< Max, ToC >= 0, ToC =< Max,
7   get_cell(Board, ToR, ToC, '.').
8
9 valid_removes(Board, P1R, P1C, P2R, P2C, Removes) :-
10  board_size(Board, Size), Max is Size - 1,
11  findall(R/C,
12    ( between(0,Max,R), between(0,Max,C),
13      \+(R:=P1R, C:=P1C), \+(R:=P2R, C:=P2C),
14      get_cell(Board, R, C, '.')),
15    Removes).
16
17 apply_move(Board, FromR, FromC, ToR, ToC, NewBoard) :-
18  get_cell(Board, FromR, FromC, Piece),
19  set_cell(Board, FromR, FromC, '.', B1),
20  set_cell(B1, ToR, ToC, Piece, NewBoard).
21
22 apply_remove(Board, Row, Col, NewBoard) :-
23  set_cell(Board, Row, Col, '#', NewBoard).
24
25 game_over(Board, Player, P1Pos, P2Pos) :-
26  player_pos(Player, P1Pos, P2Pos, CR/CC),
27  \+ valid_move(Board, CR, CC, _, _).

```

Listagem A.2: rules.pl — validação e aplicação de movimentos

### A.3 ai.pl — Algoritmo Alfa-Beta

```

1 best_move(Board, Player, P1Pos, P2Pos, Depth, MovePos, RemPos) :-
2   alphabeta(Board, Player, P1Pos, P2Pos, Depth,
3     -10000, 10000, _Val, BM, BR),
4   ( BM = none
5     -> any_legal_move(Board, Player, P1Pos, P2Pos,
6       MovePos, RemPos)
7     ; MovePos = BM, RemPos = BR
8     ).
9
10 alphabeta(Board, _Player, P1Pos, P2Pos, 0, _A, _B,
11   Value, none, none) :-
12   !, evaluate(Board, P1Pos, P2Pos, Value).
13
14 alphabeta(Board, Player, P1Pos, P2Pos, _D, _A, _B,
15   Value, none, none) :-
16   player_pos(Player, P1Pos, P2Pos, CR/CC),
17   \+ valid_move(Board, CR, CC, _, _), !,
18   terminal_value(Player, Value).
19
20 alphabeta(Board, Player, P1Pos, P2Pos, Depth, Alpha, Beta,
21   Value, BM, BR) :-
22   all_move_pairs(Board, Player, P1Pos, P2Pos, Pairs),
23   order_pairs(Pairs, Board, Player, P1Pos, P2Pos, Ordered),
24   NextDepth is Depth - 1,
25   Other is 3 - Player,

```

```
26     ( Player := 1
27     ->  ab_max(Ordered, Board, Other, P1Pos, P2Pos,
28               Player, NextDepth, Alpha, Beta,
29               -10000, none, none, Value, BM, BR)
30     ;   ab_min(Ordered, Board, Other, P1Pos, P2Pos,
31               Player, NextDepth, Alpha, Beta,
32               10000, none, none, Value, BM, BR)
33     ).
34
35 evaluate(Board, P1Pos, P2Pos, Value) :-
36     P1Pos = P1R/P1C, P2Pos = P2R/P2C,
37     valid_moves(Board, P1R, P1C, M1), length(M1, S1),
38     valid_moves(Board, P2R, P2C, M2), length(M2, S2),
39     board_size(Board, Size),
40     Center is (Size - 1) / 2.0,
41     centrality(P1R, P1C, Center, Cent1),
42     centrality(P2R, P2C, Center, Cent2),
43     Value is (S1 - S2) * 10 + (Cent1 - Cent2).
```

Listagem A.3: ai.pl — ponto de entrada e pesquisa alfa-beta

